

Documento de Desenvolvimento de Software

Sistema Web de Bolão da Copa do Mundo 2026

Stack: Next.js, Vercel, Supabase/Postgres e APIs de Futebol

Objetivo do documento. Definir escopo, requisitos, arquitetura, modelo de dados, fluxos, APIs externas, endpoints internos, estratégia de autenticação simples e plano de desenvolvimento para um site de bolão da Copa do Mundo com usuários por nome/senha e palpites salvos em JSONB no Supabase.

Versão 1.0

Data: 11 de junho de 2026

Autor: Momobubu / ChatGPT

Conteúdo

1	Resumo executivo	3
1.1	Escopo do MVP	3
1.2	Fora do escopo inicial	3
2	Contexto do torneio e premissas	3
3	Stack tecnológica recomendada	4
3.1	Decisão sobre Next.js: Route Handlers e Server Actions	5
4	Arquitetura geral	5
4.1	Princípios arquiteturais	5
5	Módulos do sistema	6
5.1	Estrutura de diretórios sugerida	6
6	Requisitos funcionais	7
7	Requisitos não funcionais	8
8	Modelo de dados	10
8.1	Estratégia JSONB para palpites	10
8.2	SQL inicial sugerido	11
8.3	RLS e acesso ao Supabase	13
9	Autenticação customizada	13
9.1	Cadastro	13
9.2	Login	14
10	Fluxo de salvar palpite	15
10.1	Validações do servidor	15
11	Regras de bloqueio de palpites	16
12	Regra de pontuação sugerida	16
12.1	Critérios de desempate	16
13	APIs externas necessárias	16
13.1	Dados de partidas, seleções e resultados	17
13.2	Recomendação de escolha	17
13.3	Endpoints externos esperados	17
13.4	Bandeiras	18
14	Pipeline de sincronização	19
14.1	Cadência sugerida	19
14.2	Proteção do endpoint de sync	20
15	Endpoints internos	20

16 Serviços de domínio	21
16.1 AuthService	21
16.2 WorldCupService	21
16.3 PredictionService	21
16.4 ScoringService	21
17 Validação de dados com Zod	21
18 Interface e telas	22
18.1 Telas do MVP	22
18.2 Componentes principais	22
19 Fluxos principais	23
19.1 Fluxo de cadastro/login	23
19.2 Fluxo de palpite	23
19.3 Fluxo de ranking	23
20 Segurança	23
21 Plano de implementação	24
21.1 Fase 1 - Base do projeto	24
21.2 Fase 2 - Auth custom	24
21.3 Fase 3 - Dados da Copa	24
21.4 Fase 4 - Palpites	24
21.5 Fase 5 - Pontuação e ranking	25
21.6 Fase 6 - Polimento	25
22 Backlog inicial	25
23 Testes e critérios de aceite	25
23.1 Casos de teste essenciais	26
24 Observabilidade e operação	26
24.1 Logs mínimos	26
24.2 Alertas manuais para MVP	26
25 Roadmap pós-MVP	27
26 Referências técnicas consultadas	27
A Checklist de entrega do MVP	27
B Arquivos Mermaid incluídos no pacote	28

1 Resumo executivo

Este documento propõe um sistema web de bolão da Copa do Mundo executando em **Next.js** e hospedado na **Vercel**, com persistência em **Supabase/Postgres**. O sistema terá cadastro simples por **nome de usuário e senha**, sem usar Supabase Auth, e armazenará os palpites dos usuários em uma coluna **JSONB** associada ao usuário. A aplicação também manterá um cache interno dos dados da Copa, vindo de uma API de futebol, para evitar dependência direta e excessiva de chamadas externas.

Decisão técnica

A decisão principal é separar três responsabilidades: **produto** (bolão, ranking, palpites), **dados oficiais** (fixtures, resultados e seleções vindos de API externa) e **identidade simples** (usuário/senha customizados com sessão própria). Isso mantém o MVP simples sem sacrificar segurança básica.

1.1 Escopo do MVP

O MVP deve permitir:

- cadastro e login por nome de usuário e senha;
- listagem dos jogos da Copa com datas, fases, seleções e placar oficial quando disponível;
- cadastro e edição de palpites antes do bloqueio de cada partida;
- armazenamento dos palpites como JSONB por usuário;
- cálculo de pontuação após resultados oficiais;
- ranking geral dos participantes;
- sincronização periódica dos dados da Copa via API externa;
- exibição de bandeiras das seleções.

1.2 Fora do escopo inicial

- pagamento, premiação financeira, carteira, apostas reais ou qualquer mecânica de gambling;
- login social, e-mail transacional e recuperação automática de senha;
- painel administrativo complexo com múltiplas ligas privadas;
- chat em tempo real;
- app mobile nativo.

2 Contexto do torneio e premissas

A Copa do Mundo de 2026 é utilizada como torneio-alvo. A aplicação deve considerar o formato expandido com 48 seleções, 104 partidas e fase de grupos com 12 grupos de 4 equipes. A arquitetura deve ser suficientemente genérica

para aceitar outros torneios no futuro, mas sem aumentar a complexidade do MVP.

Nota

Como os dados de jogos e resultados mudam durante o torneio, a aplicação não deve depender de informações hardcoded. O recomendado é manter um cache interno no banco e atualizar esse cache por jobs de sincronização.

3 Stack tecnológica recomendada

Camada	Tecnologia	Justificativa
Frontend e backend web	Next.js com App Router e TypeScript	Permite Server Components, Route Handlers e Server Actions no mesmo projeto, reduzindo boilerplate e mantendo segredos no servidor.
Hospedagem	Vercel	Deploy simples para Next.js, preview deployments e cron jobs para sincronização.
Banco de dados	Supabase Postgres	Postgres gerenciado, JSONB, índices, SQL, backups e boa integração com aplicações serverless.
Persistência flexível	JSONB	Ideal para salvar o conjunto de palpites do usuário em um documento único, validando com schema no servidor.
UI	Tailwind CSS + shadcn/ui opcional	Criação rápida de interface limpa, responsiva e com componentes reutilizáveis.
Validação	Zod	Validação de payloads, formulário de login, schema de palpites e variáveis de ambiente.
Hash de senha	Argon2id ou bcrypt	Senhas nunca devem ser salvas em texto puro. O hash deve ser calculado apenas no servidor.
APIs de futebol	Sportmonks, API-Football ou football-data.org	Fornecem fixtures, resultados, equipes, standings e dados de partida.

Bandeiras	FlagCDN + ISO2 local; Countries apoio	tabela REST como	Permite renderizar bandeiras via CDN usando códigos ISO 3166-1 alpha-2.
-----------	--	------------------------	---

3.1 Decisão sobre Next.js: Route Handlers e Server Actions

Use **Server Actions** para mutações chamadas diretamente pela UI, como salvar palpite, login e logout. Use **Route Handlers** para endpoints HTTP explícitos, como sync admin, webhooks, healthcheck e APIs consumidas por scripts.

Recomendação

Regra prática: se a ação nasce de um formulário da própria aplicação, use Server Action. Se a rota precisa ser chamada por cron, integração externa ou cliente HTTP independente, use Route Handler.

4 Arquitetura geral

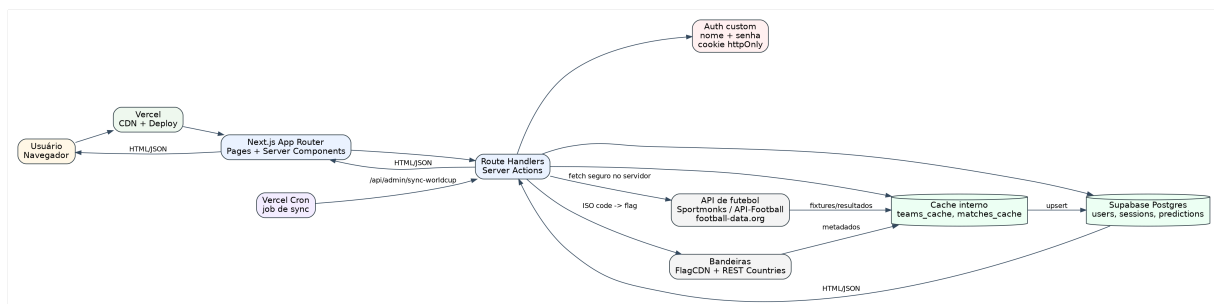


Figura 1: Arquitetura geral do sistema de bolão.

4.1 Princípios arquiteturais

1. **Segredos só no servidor:** tokens de API, chave service role do Supabase e segredo de sessão nunca vão para o browser.
2. **Banco como fonte da aplicação:** a UI lê dados normalizados/cacheados no Supabase, não diretamente da API externa.
3. **Palpites editáveis até o bloqueio:** o servidor valida se a partida ainda aceita alterações.
4. **Pontuação idempotente:** recalcular ranking várias vezes deve produzir o mesmo resultado.
5. **MVP simples, crescimento possível:** JSONB para palpites agora; tabelas auxiliares permitem otimizar ranking depois.

5 Módulos do sistema

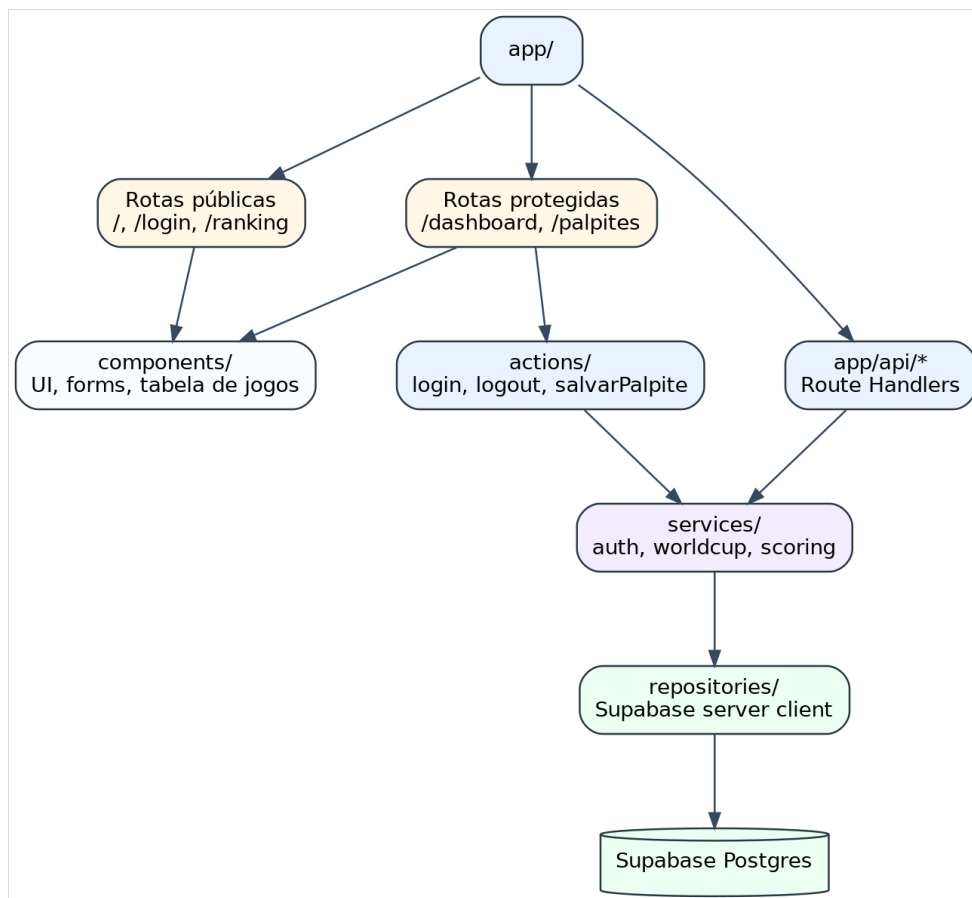


Figura 2: Organização sugerida dos módulos no projeto Next.js.

5.1 Estrutura de diretórios sugerida

```

/app
  /(public)
    page.tsx
    login/page.tsx
    ranking/page.tsx
  /(protected)
    dashboard/page.tsx
    palpites/page.tsx
  /api
    /health/route.ts
    /admin/sync-worldcup/route.ts
    /ranking/route.ts
/actions
  auth.actions.ts
  predictions.actions.ts
/components
  ui/
  match-card.tsx
    
```

```
prediction-form.tsx
ranking-table.tsx
/lib
  env.ts
  supabase-server.ts
  session.ts
/services
  auth.service.ts
  worldcup.service.ts
  scoring.service.ts
  flags.service.ts
/repositories
  users.repo.ts
  predictions.repo.ts
  matches.repo.ts
  ranking.repo.ts
/types
  prediction.ts
  worldcup.ts
```

Listing 1: Estrutura inicial do projeto

6 Requisitos funcionais

ID	Requisito funcional	Prioridade
RF-01	O sistema deve permitir cadastro de usuário por nome único e senha.	Obrigatório
RF-02	O sistema deve permitir login e logout com sessão persistida em cookie HTTP-only.	Obrigatório
RF-03	O sistema deve listar partidas da Copa com fase, data, horário, seleções, bandeiras e status.	Obrigatório
RF-04	O sistema deve permitir que o usuário salve ou edite palpites antes do bloqueio da partida.	Obrigatório
RF-05	O sistema deve armazenar os palpites em JSONB atrelado ao usuário e ao torneio.	Obrigatório
RF-06	O sistema deve impedir alteração de palpite quando a partida já estiver bloqueada.	Obrigatório
RF-07	O sistema deve sincronizar dados da Copa via API externa em rotina agendada.	Obrigatório
RF-08	O sistema deve calcular pontuação dos usuários conforme regra definida.	Obrigatório

RF-09	O sistema deve exibir ranking geral ordenado por pontuação e critérios de desempate.	Obrigatório
RF-10	O sistema deve permitir visualização dos próprios palpites salvos.	Obrigatório
RF-11	O sistema deve permitir recalcular ranking manualmente via endpoint protegido de admin.	Desejável
RF-12	O sistema deve exibir histórico simples de atualização dos dados externos.	Desejável
RF-13	O sistema deve ter página de healthcheck para verificar ambiente e banco.	Desejável
RF-14	O sistema deve suportar múltiplos torneios no modelo de dados, ainda que o MVP use apenas WC2026.	Futuro

7 Requisitos não funcionais

ID	Requisito não funcional	Critério
RNF-01	Segurança de senha	Senhas armazenadas somente com hash Argon2id ou bcrypt; nunca texto puro.
RNF-02	Sessão segura	Cookie HTTP-only, Secure em produção, SameSite=Lax, expiração e hash de token no banco.
RNF-03	Segredos protegidos	Tokens de APIs e service role do Supabase somente em variáveis de ambiente no servidor.

RNF-04	Disponibilidade	A aplicação deve continuar listando dados já cacheados se a API externa falhar.
RNF-05	Performance	Ranking e listagem de jogos devem carregar usando dados do banco/cache, evitando fan-out externo.
RNF-06	Observabilidade	Logs mínimos para login falho, sync de API, erro de cálculo e update de palpite.
RNF-07	Escalabilidade	O modelo deve suportar ao menos centenas de usuários sem mudança estrutural.
RNF-08	Manutenibilidade	Código com services/repositories separados, tipagem forte e validação de entrada.
RNF-09	Portabilidade	Banco baseado em Postgres padrão, reduzindo lock-in.
RNF-10	Acessibilidade	Formulários utilizáveis por teclado, contraste adequado e labels nos inputs.
RNF-11	Consistência temporal	Todas as datas de partidas devem ser salvas em UTC e exibidas no fuso do usuário.

RNF-12 Rate limit externo

Sync deve respeitar limite do provedor e usar cache para evitar chamadas desnecessárias.

8 Modelo de dados

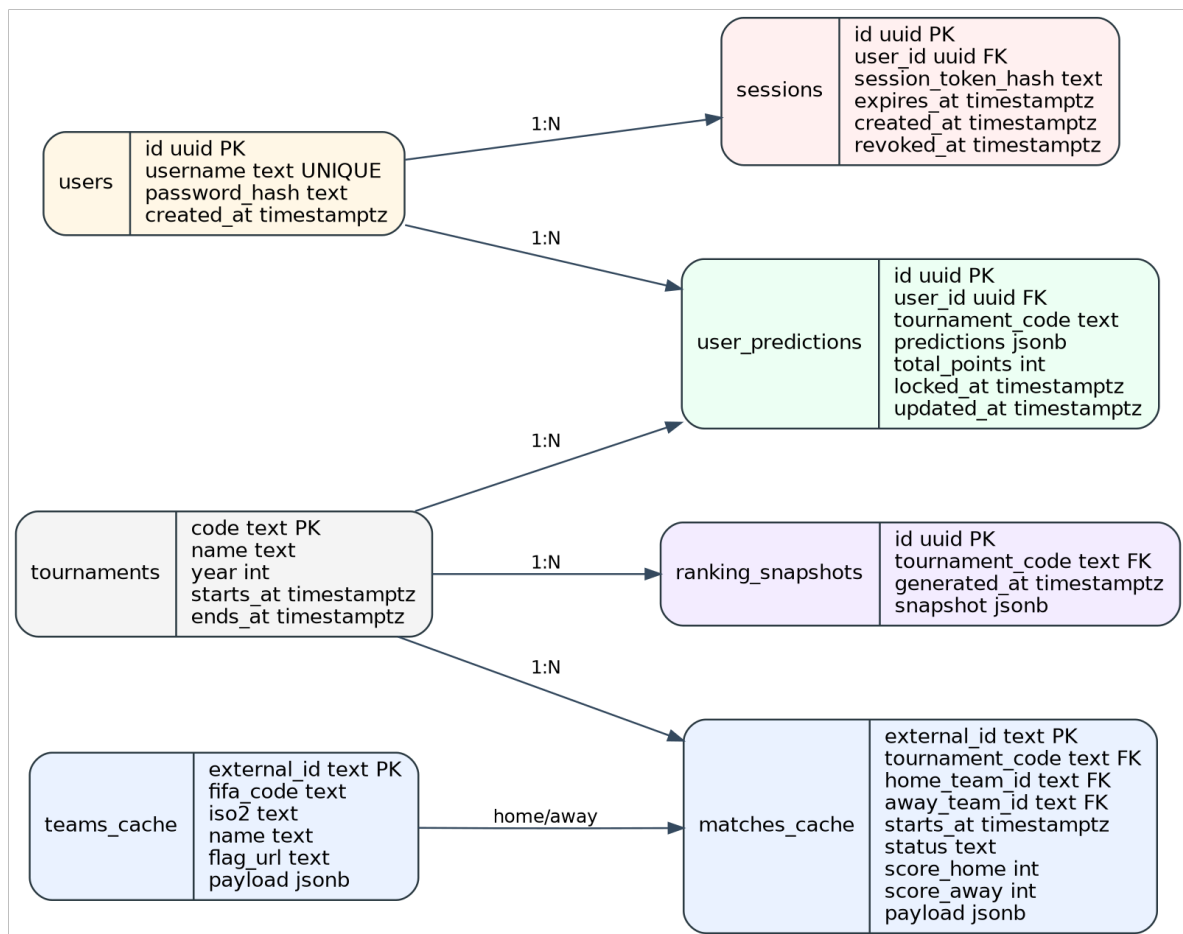


Figura 3: Modelo de dados relacional com JSONB para palpites.

8.1 Estratégia JSONB para palpites

A ideia do usuário ter “um JSON” é válida para o MVP, desde que esse JSON fique em uma tabela própria e seja validado pelo servidor. A proposta é a tabela `user_predictions`, com uma linha por usuário e torneio. A coluna `predictions` armazena o documento de palpites.

```
{
  "version": 1,
  "tournamentCode": "WC2026",
  "updatedAt": "2026-06-11T12:00:00.000Z",
```

```

"matches": {
  "match_001": {
    "homeTeamId": "BRA",
    "awayTeamId": "GER",
    "homeGoals": 2,
    "awayGoals": 1,
    "savedAt": "2026-06-11T12:00:00.000Z",
    "locked": false,
    "points": null
  },
  "match_002": {
    "homeTeamId": "ARG",
    "awayTeamId": "FRA",
    "homeGoals": 1,
    "awayGoals": 1,
    "savedAt": "2026-06-11T12:05:00.000Z",
    "locked": false,
    "points": null
  }
},
"summary": {
  "totalPoints": 0,
  "exactScores": 0,
  "correctOutcomes": 0,
  "lastCalculatedAt": null
}
}

```

Listing 2: Formato proposto do JSON de palpites**Risco / cuidado**

Evite salvar tudo em uma coluna JSON dentro da própria tabela users. Isso mistura identidade com dado de domínio. Melhor manter users apenas para login e criar user_predictions para os palpites.

8.2 SQL inicial sugerido

```

create extension if not exists pgcrypto;

create table if not exists users (
  id uuid primary key default gen_random_uuid(),
  username text not null unique,
  password_hash text not null,
  created_at timestampz not null default now(),
  updated_at timestampz not null default now()
);

create table if not exists sessions (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null references users(id) on delete cascade,

```

```

    session_token_hash text not null unique,
    user_agent text,
    ip_hash text,
    expires_at timestamptz not null,
    revoked_at timestamptz,
    created_at timestamptz not null default now()
);

create table if not exists tournaments (
    code text primary key,
    name text not null,
    year int not null,
    starts_at timestamptz,
    ends_at timestamptz,
    created_at timestamptz not null default now()
);

create table if not exists user_predictions (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references users(id) on delete cascade,
    tournament_code text not null references tournaments(code),
    predictions jsonb not null default '{"version":1,"matches":{},"summary":{}}'::jsonb,
    total_points int not null default 0,
    exact_scores int not null default 0,
    correct_outcomes int not null default 0,
    updated_at timestamptz not null default now(),
    locked_at timestamptz,
    unique(user_id, tournament_code)
);

create table if not exists teams_cache (
    external_id text primary key,
    fifa_code text,
    iso2 text,
    name text not null,
    flag_url text,
    payload jsonb not null default '{}'::jsonb,
    updated_at timestamptz not null default now()
);

create table if not exists matches_cache (
    external_id text primary key,
    tournament_code text not null references tournaments(code),
    home_team_id text references teams_cache(external_id),
    away_team_id text references teams_cache(external_id),
    starts_at timestamptz not null,
    stage text,
    group_name text,
    status text not null,
    score_home int,
    score_away int,
    payload jsonb not null default '{}'::jsonb,

```

```

    updated_at timestamptz not null default now()
  );

create table if not exists ranking_snapshots (
  id uuid primary key default gen_random_uuid(),
  tournament_code text not null references tournaments(code),
  generated_at timestamptz not null default now(),
  snapshot jsonb not null
);

create index if not exists idx_sessions_user_id on sessions(user_id);
create index if not exists idx_sessions_expires_at on sessions(
  expires_at);
create index if not exists idx_predictions_user_tournament on
  user_predictions(user_id, tournament_code);
create index if not exists idx_predictions_jsonb on user_predictions
  using gin(predictions);
create index if not exists idx_matches_tournament_start on
  matches_cache(tournament_code, starts_at);
create index if not exists idx_matches_status on matches_cache(status);

```

Listing 3: DDL inicial para Supabase/Postgres

8.3 RLS e acesso ao Supabase

Como o projeto não usará Supabase Auth, há duas opções:

1. **Opção recomendada para MVP:** nunca consultar Supabase direto do browser. Todas as consultas passam pelo servidor Next.js usando variáveis de ambiente privadas. Nesse caso, o service role fica apenas no servidor.
2. **Opção avançada:** criar uma camada de policies/RLS customizada com JWT próprio. É mais seguro para acesso direto do cliente, mas aumenta bastante a complexidade.

Recomendação

Para este projeto, use Supabase apenas no servidor. No frontend, chame Server Actions ou Route Handlers do Next.js. Isso permite autenticação custom sem expor credenciais sensíveis no cliente.

9 Autenticação customizada

O sistema não utilizará Supabase Auth. Ainda assim, segurança mínima é obrigatória.

9.1 Cadastro

Fluxo de cadastro:

1. usuário informa username e password;
2. servidor valida tamanho, caracteres e unicidade do nome;
3. servidor gera hash da senha com Argon2id ou bcrypt;
4. servidor salva username e password_hash;
5. servidor cria sessão e cookie HTTP-only.

9.2 Login

Fluxo de login:

1. servidor busca usuário pelo nome;
2. compara senha recebida com password_hash;
3. se válido, gera token aleatório de sessão;
4. salva hash(token) na tabela sessions;
5. envia cookie com o token puro para o navegador;
6. em cada requisição protegida, o servidor hasheia o token do cookie e compara com o banco.

Risco / cuidado

Nunca salve senha em texto puro. Mesmo em projeto pequeno, vazamento de banco com senha pura é uma falha grave. Também não salve token de sessão puro no banco; salve hash do token.

10 Fluxo de salvar palpite

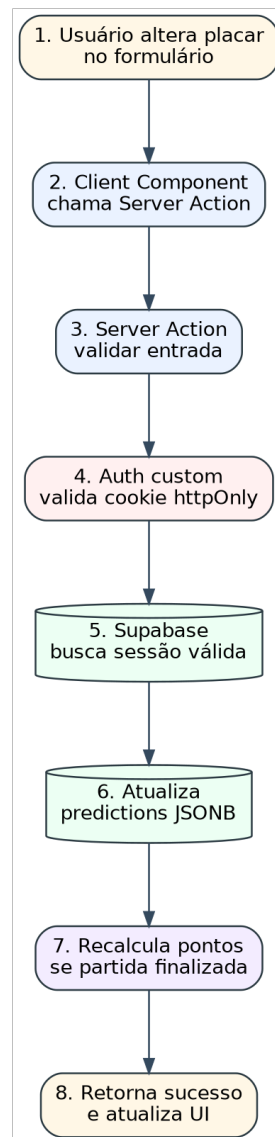


Figura 4: Fluxo simplificado de edição e persistência de palpite.

10.1 Validações do servidor

Ao salvar um palpite, o servidor deve validar:

- sessão existe, não expirou e não foi revogada;
- partida existe no cache interno;
- partida ainda não começou ou ainda não atingiu o horário de bloqueio configurado;
- gols informados são inteiros entre 0 e 30;
- o JSON resultante respeita o schema esperado;
- o usuário só edita o próprio documento de palpites.

11 Regras de bloqueio de palpites

Regra	Definição
Bloqueio padrão	Uma partida bloqueia 5 minutos antes de <code>starts_at</code> .
Partida em andamento	Se <code>status</code> for <code>IN_PLAY</code> , <code>LIVE</code> , <code>PAUSED</code> ou equivalente, não aceita palpite.
Partida finalizada	Se <code>status</code> for <code>FINISHED</code> , não aceita palpite e já pode pontuar.
Jogo adiado	Se a API alterar <code>starts_at</code> , o sistema usa o novo horário salvo no cache.
Dados ausentes	Se não houver horário confiável, bloquear manualmente ou manter indisponível até <code>sync</code> correto.

12 Regra de pontuação sugerida

Acerto	Pontos	Exemplo
Placar exato	5	Palpite 2x1, resultado 2x1.
Resultado correto sem placar exato	3	Palpite 2x1, resultado 1x0.
Empate correto sem placar exato	3	Palpite 1x1, resultado 0x0.
Acerto de gols de uma seleção	1 bônus	Palpite 2x0, resultado 2x1.
Erro total	0	Palpite vitória, resultado derrota.

12.1 Critérios de desempate

1. maior pontuação total;
2. maior número de placares exatos;
3. maior número de resultados corretos;
4. menor quantidade de palpites em branco;
5. usuário criado há mais tempo, se ainda houver empate.

13 APIs externas necessárias

13.1 Dados de partidas, seleções e resultados

API	Uso no projeto	Observação técnica
Sportmonks Football API	Fixtures, livescores, standings, bracket, squads e eventos.	Boa cobertura para Copa 2026, dados em JSON, filtros e includes. É opção forte se aceitar plano pago.
API-Football / API-Sports	Fixtures, teams, standings, livescore, lineups e eventos.	Para Copa 2026, a documentação/guia usa league=1 e season=2026. Boa opção para MVP com conta de API.
football-data.org	Competição WC, jogos, equipes e standings.	Mais simples; verificar plano e limite. Bom fallback se dados necessários estiverem disponíveis no plano escolhido.
FIFA oficial	Conferência manual de calendário/resultados.	Não tratar como API pública estável para app; usar como referência visual/manual, não como backend por scraping.

13.2 Recomendação de escolha

Decisão técnica

Para MVP, escolha **um** provedor primário: API-Football ou Sportmonks. Implemente o código com interface `FootballProvider`, para trocar de provedor depois sem refatorar o produto inteiro. Use `football-data.org` como alternativa/fallback se o plano atender aos dados de World Cup.

13.3 Endpoints externos esperados

Os nomes exatos variam por provedor, mas o sistema precisa das seguintes capacidades:

Capacidade	Dados necessários	Exemplo de uso interno
Fixtures/lista de jogos	ID externo, mandante, visitante, data UTC, estádio, fase, grupo.	Popular matches_cache.
Resultados	Status, placar final, minutos, vencedor.	Calcular pontuação e bloquear palpites.
Seleções/equipes	ID externo, nome, código FIFA, país, logo se houver.	Popular teams_cache.

Standings/grupos	Grupo, pontos, vitórias, saldo, posição.	Exibir tabela do grupo e validar fase de mata-mata.
Liveness	Jogos em andamento e eventos.	Atualizar placares durante o dia de jogo.
Bracket/mata-mata	Chaveamento, próximos confrontos, vencedores.	Exibir oitavas/quartas/semi/final.

13.4 Bandeiras

Para bandeiras, a opção mais simples é usar **FlagCDN**, que fornece imagens por código ISO 3166-1 alpha-2. O sistema deve manter no banco uma coluna `iso2` por seleção e gerar a URL de bandeira, por exemplo:

```
https://flagcdn.com/w80/br.png
https://flagcdn.com/w80/ar.png
https://flagcdn.com/w80/fr.png
```

Listing 4: Exemplo de URL de bandeira

Para mapear nome/código de seleção para `iso2`, pode-se manter uma tabela local curada. REST Countries pode ajudar com metadados de países, mas seleções de futebol nem sempre mapeiam perfeitamente para países ISO por nome usado pela FIFA. Por isso, a tabela local deve ser a fonte final.

Nota

Exemplo de casos que exigem cuidado: England, Scotland, Wales e Northern Ireland têm seleções próprias no futebol, mas nem todos aparecem como países ISO independentes da mesma forma que seleções FIFA. Para Copa 2026 masculina, a lista de participantes deve ser mapeada manualmente no seed inicial.

14 Pipeline de sincronização

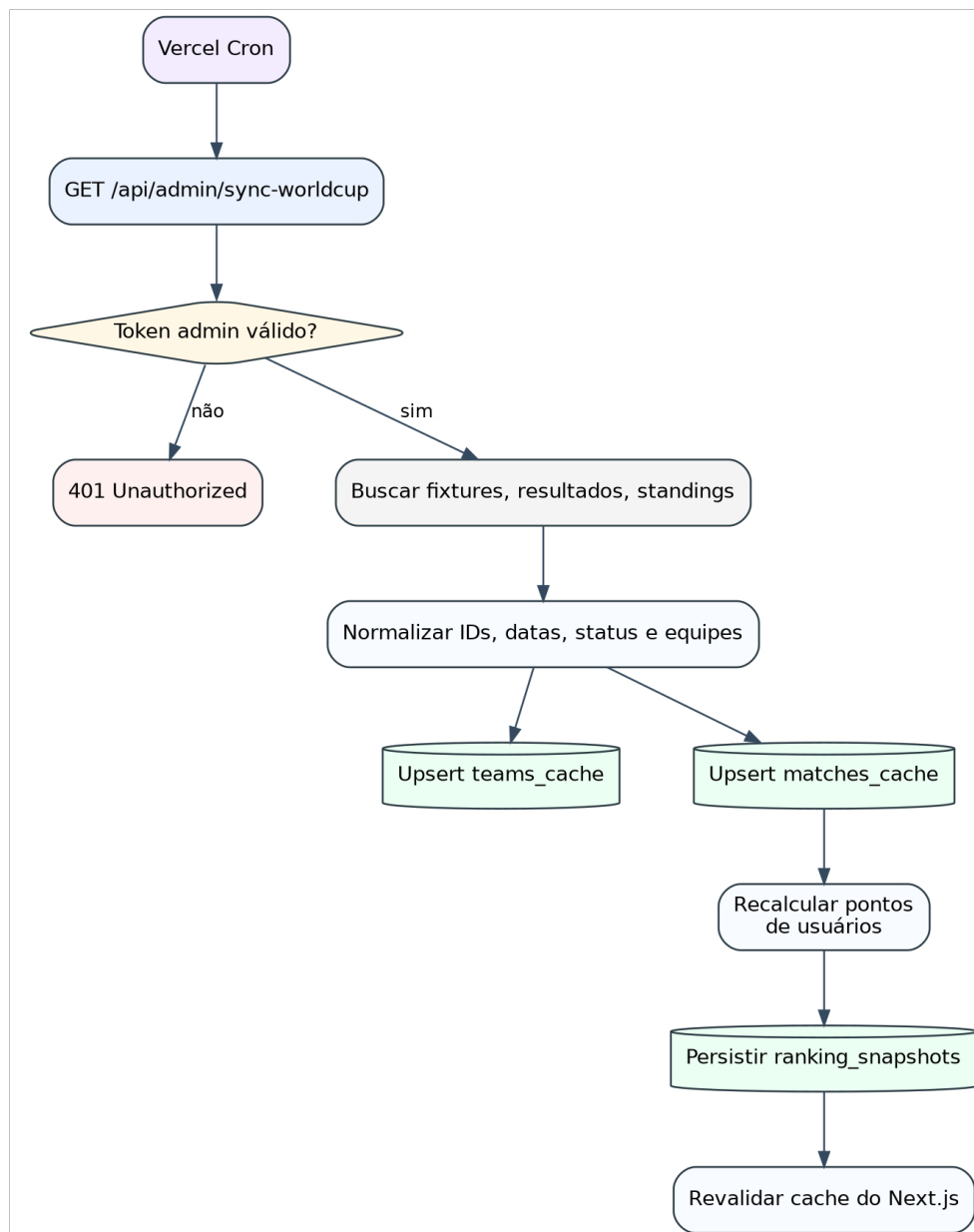


Figura 5: Pipeline de sincronização de dados da Copa.

14.1 Cadência sugerida

Período	Frequência	Justificativa
Antes da Copa	1 a 2 vezes por dia	Calendário e equipes mudam pouco.
Dia sem jogo	A cada 6 horas	Garante ajustes de tabela e datas sem gastar quota.

Dia com jogo	A cada 5 a 15 minutos	Atualiza status e placares com boa experiência.
Durante jogo ao vivo	A cada 1 a 3 minutos, se o plano permitir	Só se o ranking ao vivo for importante.
Após finalização	Rodar sync e recalcular ranking	Garante pontuação correta.

14.2 Proteção do endpoint de sync

O endpoint `/api/admin/sync-worldcup` deve exigir um segredo administrativo enviado por header, por exemplo `x-admin-sync-token`. Esse segredo fica em variável de ambiente na Vercel.

```
NEXT_PUBLIC_APP_URL=https://seu-bolao.vercel.app
SUPABASE_URL=https://xxxxx.supabase.co
SUPABASE_SERVICE_ROLE_KEY=...
SESSION_SECRET=...
FOOTBALL_PROVIDER=api-football
FOOTBALL_API_KEY=...
ADMIN_SYNC_TOKEN=...
```

Listing 5: Variáveis de ambiente esperadas

15 Endpoints internos

Endpoint/Ação	Método	Responsabilidade	Auth
loginAction	Server Action	Validar credenciais, criar sessão e cookie.	Não
logoutAction	Server Action	Revogar sessão e limpar cookie.	Sim
savePredictionAction	Server Action	Validar partida e atualizar JSONB de palpites.	Sim
/api/health	GET	Conferir disponibilidade básica da aplicação.	Não
/api/ranking	GET	Retornar ranking cacheado ou calculado.	Não/Sim
/api/admin/sync-worldcup	GET/POST	Buscar dados externos e atualizar cache.	Admin
/api/admin/recalculate-ranking	POST	Reprocessar pontuação de todos os usuários.	Admin

16 Serviços de domínio

16.1 AuthService

Responsabilidades:

- normalizar e validar username;
- gerar e verificar hash de senha;
- criar, consultar e revogar sessões;
- ler usuário atual a partir do cookie;
- bloquear tentativa de login em excesso.

16.2 WorldCupService

Responsabilidades:

- consumir o provedor externo;
- normalizar partidas e seleções;
- preencher cache local;
- resolver bandeiras e metadados;
- oferecer consultas para a UI.

16.3 PredictionService

Responsabilidades:

- validar payload de palpite;
- verificar regra de bloqueio;
- atualizar JSONB de palpites;
- criar documento inicial de palpites do usuário;
- expor resumo dos palpites do usuário.

16.4 ScoringService

Responsabilidades:

- comparar palpites com resultados oficiais;
- aplicar regra de pontuação;
- atualizar total_points, exact_scores e correct_outcomes;
- gerar snapshots de ranking;
- garantir idempotência.

17 Validação de dados com Zod

```
const PredictionInputSchema = z.object({
  matchId: z.string().min(1),
  homeGoals: z.number().int().min(0).max(30),
  awayGoals: z.number().int().min(0).max(30),
```

```
});

const PredictionDocumentSchema = z.object({
  version: z.literal(1),
  tournamentCode: z.string(),
  updatedAt: z.string().datetime(),
  matches: z.record(z.string(), z.object({
    homeTeamId: z.string(),
    awayTeamId: z.string(),
    homeGoals: z.number().int().min(0).max(30),
    awayGoals: z.number().int().min(0).max(30),
    savedAt: z.string().datetime(),
    locked: z.boolean(),
    points: z.number().int().nullable(),
  })),
  summary: z.object({
    totalPoints: z.number().int(),
    exactScores: z.number().int(),
    correctOutcomes: z.number().int(),
    lastCalculatedAt: z.string().datetime().nullable(),
  }),
});
```

Listing 6: Schema conceitual de palpite em TypeScript/Zod

18 Interface e telas

18.1 Telas do MVP

Tela	Conteúdo
Home	Chamada do bolão, botão de login/cadastro, explicação da pontuação e link para ranking.
Login/Cadastro	Formulário simples com nome de usuário e senha.
Dashboard	Resumo do usuário, pontuação atual, próximos jogos e alertas de palpites pendentes.
Palpites	Lista de jogos agrupada por data/fase, inputs de placar, bandeiras e status de bloqueio.
Ranking	Tabela ordenada com posição, usuário, pontos, placares exatos e acertos.
Admin Sync	Opcional no MVP: tela protegida ou endpoint para disparar sincronização e recalculer ranking.

18.2 Componentes principais

- MatchCard: mostra uma partida, bandeiras, horário, status e inputs de placar.

- PredictionForm: gerencia envio de palpite com estado pendente e mensagens de erro.
- RankingTable: tabela de classificação do bolão.
- UserSummary: pontuação e estatísticas rápidas do usuário.
- StageFilter: filtros por fase, grupo e data.

19 Fluxos principais

19.1 Fluxo de cadastro/login

1. Usuário acessa /login.
2. Informa nome e senha.
3. Server Action valida dados.
4. Se cadastro, cria usuário; se login, valida senha.
5. Cria sessão e cookie.
6. Redireciona para /dashboard.

19.2 Fluxo de palpite

1. Usuário acessa /palpites.
2. Página busca partidas cacheadas e documento JSONB do usuário.
3. Usuário preenche placar.
4. Server Action valida sessão, partida e horário.
5. Palpite é salvo no JSONB.
6. UI mostra feedback e mantém estado atualizado.

19.3 Fluxo de ranking

1. Sync atualiza resultados no matches_cache.
2. ScoringService percorre partidas finalizadas.
3. Palpites são pontuados e resumo é atualizado.
4. Ranking é ordenado e salvo em ranking_snapshots.
5. Página /ranking exibe último snapshot.

20 Segurança

Risco	Mitigação
Senha vazada	Usar hash forte com salt, preferencialmente Argon2id.
Ataque de força bruta	Rate limit por IP/nome e atraso progressivo após falhas.

Roubo de sessão por JavaScript	Cookie HTTP-only, Secure e SameSite=Lax.
Alteração de palpite de outro usuário	Nunca confiar em userId vindo do cliente; derivar userId da sessão.
Manipulação após início do jogo	Validar bloqueio no servidor com starts_at e status cacheado.
Exposição da service role	Variável apenas no servidor; não importar cliente admin em Client Components.
Quota de API externa estourada	Cache interno, cron controlado e backoff em erro.
Dados externos inconsistentes	Normalizar payloads e preservar payload original em JSONB para auditoria.

21 Plano de implementação

21.1 Fase 1 - Base do projeto

- Criar projeto Next.js com TypeScript.
- Configurar Tailwind e componentes base.
- Criar projeto Supabase e rodar SQL inicial.
- Configurar variáveis de ambiente na Vercel.
- Implementar cliente Supabase server-side.

21.2 Fase 2 - Auth custom

- Implementar cadastro, login, logout e sessão.
- Criar middleware ou helper getCurrentUser().
- Proteger rotas de /dashboard e /palpites.
- Adicionar rate limit simples para login.

21.3 Fase 3 - Dados da Copa

- Escolher provedor primário de futebol.
- Implementar FootballProvider com interface genérica.
- Criar seed de tournaments e tabela local de mapeamento ISO2.
- Implementar sync de teams e matches.
- Criar endpoint admin protegido e cron na Vercel.

21.4 Fase 4 - Palpites

- Criar documento inicial JSONB por usuário.
- Implementar tela /palpites.
- Implementar Server Action savePredictionAction.
- Validar bloqueio por horário/status.

21.5 Fase 5 - Pontuação e ranking

- Implementar ScoringService.
- Criar rotina de recalcular ranking.
- Exibir ranking público.
- Adicionar critérios de desempate.

21.6 Fase 6 - Polimento

- Melhorar UI mobile.
- Adicionar estados de loading/erro.
- Revisar acessibilidade.
- Criar página de regras do bolão.
- Fazer testes manuais com partidas mockadas.

22 Backlog inicial

ID	História	Tipo	Prio.
US-01	Como visitante, quero criar usuário com nome e senha para participar do bolão.	Auth	Alta
US-02	Como usuário, quero fazer login para acessar meus palpites.	Auth	Alta
US-03	Como usuário, quero ver todos os jogos da Copa para preencher palpites.	Produto	Alta
US-04	Como usuário, quero salvar o placar de cada jogo para competir no ranking.	Produto	Alta
US-05	Como usuário, quero saber quais jogos já estão bloqueados.	Produto	Alta
US-06	Como usuário, quero ver minha pontuação parcial.	Produto	Média
US-07	Como visitante, quero ver ranking geral do bolão.	Produto	Alta
US-08	Como sistema, quero sincronizar jogos e resultados periodicamente.	Integração	Alta
US-09	Como admin, quero recalcular pontuação quando necessário.	Admin	Média
US-10	Como usuário, quero ver bandeiras das seleções para reconhecer os jogos rápido.	UX	Média

23 Testes e critérios de aceite

23.1 Casos de teste essenciais

ID	Cenário	Resultado esperado
CT-01	Cadastro com nome novo e senha válida.	Usuário criado, sessão ativa, redirect para dashboard.
CT-02	Cadastro com nome repetido.	Erro amigável, sem criar duplicata.
CT-03	Login com senha incorreta.	Erro, sem sessão.
CT-04	Salvar palpite antes do bloqueio.	JSONB atualizado corretamente.
CT-05	Tentar salvar palpite após início do jogo.	Operação recusada pelo servidor.
CT-06	Sync externo com sucesso.	matches_cache e teams_cache atualizados.
CT-07	API externa fora do ar.	App mantém dados cacheados e registra erro.
CT-08	Resultado final recebido.	Pontuação recalculada e ranking atualizado.
CT-09	Dois usuários empatados em pontos.	Desempate aplicado na ordem definida.
CT-10	Tentativa de editar palpite de outro usuário.	Ação negada; userId vem da sessão, não do cliente.

24 Observabilidade e operação

24.1 Logs mínimos

- início/fim do sync, provedor usado e número de partidas atualizadas;
- falhas de API externa com status code e mensagem;
- login falho sem registrar senha;
- tentativa de salvar palpite bloqueado;
- tempo de cálculo do ranking;
- data do último snapshot de ranking.

24.2 Alertas manuais para MVP

No MVP, basta uma página ou endpoint de admin que retorne:

- último sync bem-sucedido;
- último erro de sync;
- total de partidas no cache;
- total de usuários e documentos de palpites;
- data do último ranking calculado.

25 Roadmap pós-MVP

- ligas privadas com código de convite;
- bolões por grupo de amigos;
- painel admin visual;
- exportação CSV do ranking;
- palpites de campeão, artilheiro e fase mata-mata;
- notificações antes do bloqueio dos jogos;
- modo auditável com histórico de versões dos palpites;
- suporte a outros torneios.

26 Referências técnicas consultadas

- FIFA. *FIFA World Cup 2026: schedule, fixtures and stadiums*. <https://www.fifa.com/en/tournaments/mens/worldcup/canadamexicousa2026>
- Next.js. *Route Handlers*. <https://nextjs.org/docs/app/api-reference/file-conventions/route>
- Next.js. *Mutating Data / Server Functions*. <https://nextjs.org/docs/app/getting-started/mutating-data>
- Supabase. *Managing JSON and unstructured data*. <https://supabase.com/docs/guides/database/json>
- Supabase. *Row Level Security*. <https://supabase.com/docs/guides/database/postgres/row-level-security>
- API-Football / API-Sports. *FIFA World Cup 2026 guide*. <https://www.api-football.com/news/post/fifa-world-cup-2026-guide-to-using-data-with-api-sports>
- Sportmonks. *World Cup 2026 API*. <https://www.sportmonks.com/football-api/world-cup-api/>
- football-data.org. *API Reference*. <https://www.football-data.org/documentation/api>
- FlagCDN. *Flags API and CDN*. <https://flagcdn.com/>
- REST Countries. *Country data API*. <https://restcountries.com/>

A Checklist de entrega do MVP

Item	Status
Projeto Next.js criado com TypeScript.	☐
Supabase configurado com SQL inicial.	☐
Variáveis de ambiente configuradas localmente e na Vercel.	☐
Cadastro/login/logout implementados.	☐
Sessão com cookie HTTP-only implementada.	☐
Sync de partidas funcionando com provedor escolhido.	☐
Cache de seleções e bandeiras preenchido.	☐
Tela de palpites funcionando.	☐
Bloqueio por horário/status funcionando.	☐
Pontuação e ranking funcionando.	☐
Deploy na Vercel funcionando.	☐
Cron de sync configurado.	☐
Testes manuais principais concluídos.	☐

B Arquivos Mermaid incluídos no pacote

Os diagramas deste documento foram incluídos como imagens compiladas na pasta `diagramas/`. O pacote-fonte também contém os arquivos Mermaid em `mermaid/`, para edição futura em ferramentas compatíveis com Mermaid.

- `arquitetura_geral.mmd`
- `er_model.mmd`
- `fluxo_login_palpite.mmd`
- `sync_pipeline.mmd`
- `componentes_next.mmd`